

RAPPORT DE PROJET

Application Mobile de Gestion de Mots de Passe

avec Firebase & Flutter

Licence 3 — Informatique

Projet individuel

Année universitaire 2025 — 2026

1. Présentation du Projet

1.1 Contexte et Objectifs

Dans le cadre de notre formation en Licence 3 Informatique, ce projet a consisté en la conception et le développement d'une application mobile de gestion de mots de passe. Face à la multiplication des services numériques et à la nécessité de sécuriser les identifiants, ce gestionnaire répond à un besoin concret et quotidien.

L'objectif principal était de développer une application mobile fonctionnelle intégrant une base de données cloud, une authentification sécurisée, ainsi qu'un système de notifications push, le tout dans un délai d'un mois.

1.2 Périmètre du Projet

- Conception d'une interface utilisateur intuitive et moderne
- Stockage sécurisé des mots de passe dans Firebase Firestore
- Authentification des utilisateurs via Firebase Authentication
- Intégration des notifications push via Firebase Cloud Messaging (FCM)
- Tests sur émulateur Android via Android Studio

2. Architecture Technique

2.1 Stack Technologique

L'application repose sur un ensemble de technologies modernes et éprouvées permettant un développement rapide et une maintenance facilitée.

Technologie	Rôle
Flutter (Dart)	Framework principal de développement mobile cross-platform
Firebase Core	Initialisation et configuration de l'écosystème Firebase
Firebase Auth	Authentification sécurisée des utilisateurs
Cloud Firestore	Base de données NoSQL en temps réel
Firebase Messaging	Envoi et réception de notifications push (FCM)
Android Studio	Émulateur Android pour les tests
VS Code	Environnement de développement principal

2.2 Structure du Projet

Le projet suit une architecture en couches, séparant clairement la logique métier, les données et la présentation :

- lib/models/ — Modèles de données de l'application
- lib/screens/ — Interfaces utilisateur (pages)
- lib/services/ — Services métier et accès aux données Firebase
- lib/firebase_options.dart — Configuration Firebase générée automatiquement
- android/app/ — Configuration Android incluant google-services.json

2.2(bis) Structure du Projet

Le projet compte plusieurs pages dont :

- etudiant_screen.dart
- mot_de_passe.dart
- motdepasseoublie.dart
- page_de_connexion.dart
- settings.dart

2.3 Flux de Communication FCM

Les notifications push suivent le flux suivant : depuis la Firebase Console, un message est envoyé au serveur FCM de Google, qui le transmet à l'appareil cible identifié par son token d'enregistrement unique. Le SDK Firebase Messaging Flutter réceptionne ensuite le message et déclenche les handlers appropriés selon l'état de l'application (premier plan, arrière-plan ou fermée).

3. Fonctionnalités de l'Application

3.1 Authentification

L'application intègre un système d'authentification complet via Firebase Authentication. Les utilisateurs peuvent créer un compte, se connecter, et réinitialiser leur mot de passe en cas d'oubli. L'accès aux données est conditionné à une session active, garantissant la confidentialité des informations.

- Page de connexion avec email et mot de passe
- Création de compte utilisateur
- Récupération de mot de passe oublié par email
- Déconnexion sécurisée

3.2 Gestion des Mots de Passe

Le cœur de l'application permet de stocker, consulter et gérer ses identifiants de manière centralisée. Chaque entrée est liée au compte utilisateur connecté, garantissant l'isolation des données entre les différents utilisateurs.

- Ajout de nouveaux mots de passe avec titre, identifiant et valeur
- Consultation de la liste des mots de passe enregistrés
- Stockage sécurisé dans Cloud Firestore

3.3 Notifications Push (FCM)

L'intégration de Firebase Cloud Messaging permet d'envoyer des notifications push à l'application. À l'initialisation, l'application génère un token FCM unique identifiant l'appareil. Ce token est utilisé pour cibler des notifications spécifiques depuis la Firebase Console ou depuis un serveur backend.

- Demande de permission de notifications au démarrage
- Génération et récupération du token FCM de l'appareil
- Réception des notifications en premier plan (foreground) avec affichage dans les logs
- Réception des notifications en arrière-plan (background) avec affichage système

3.4 Paramètres

Une page de paramètres permet à l'utilisateur de personnaliser son expérience et de gérer son compte au sein de l'application.

4. Intégration Firebase et FCM

4.1 Configuration Firebase

La configuration Firebase a été réalisée via l'outil FlutterFire CLI, qui génère automatiquement le fichier `firebase_options.dart` contenant les paramètres de connexion pour chaque plateforme. Le fichier `google-services.json` a été intégré dans le répertoire `android/app/` pour permettre la communication avec les services Google.

4.2 Initialisation dans l'Application

L'initialisation de Firebase est effectuée de manière asynchrone au démarrage de l'application, avant le lancement de l'interface. Un handler dédié gère les messages reçus lorsque l'application est en arrière-plan, annoté avec `@pragma('vm:entry-point')` pour garantir son exécution dans un isolat Dart séparé.

4.3 Cycle de Vie des Notifications

Le comportement de l'application face aux notifications varie selon son état :

État de l'app	Comportement
Premier plan (Foreground)	Notification interceptée par <code>onMessage.listen()</code> , gérée dans le code Dart
Arrière-plan (Background)	Notification affichée automatiquement par le système Android
Application fermée (Terminated)	Handler background déclenché à la réouverture de l'application

5. Conclusion et Perspectives

5.1 Bilan du Projet

Ce projet a permis de développer en moins d'un mois une application mobile complète et fonctionnelle, intégrant des services cloud modernes. Les objectifs initiaux ont été atteints : l'application offre une gestion sécurisée des mots de passe, une authentification robuste via Firebase, et un système de notifications push opérationnel via FCM.

Ce projet a été l'occasion d'approfondir mes connaissances en développement mobile cross-platform avec Flutter et Dart, ainsi que d'acquérir une expérience concrète sur l'écosystème Firebase, notamment la configuration des fichiers Gradle, la gestion des tokens FCM et le cycle de vie des notifications Android.

5.2 Difficultés Rencontrées

- Configuration initiale de Firebase avec Flutter nécessitant une compréhension des fichiers Gradle
- Gestion du cycle de vie des notifications selon l'état de l'application (foreground/background)
- Compréhension du fonctionnement des tokens FCM et de leur génération sur émulateur
- Configuration de l'émulateur Android avec Google Play Services pour supporter FCM

5.3 Perspectives d'Évolution

Plusieurs améliorations pourraient être envisagées pour faire évoluer l'application :

- Chiffrement côté client des mots de passe avant stockage dans Firestore
- Ajout du package flutter_local_notifications pour afficher les notifications en premier plan
- Implémentation d'une authentification biométrique (empreinte digitale, Face ID)
- Génération automatique de mots de passe forts
- Export sécurisé des données et synchronisation multi-appareils
- Déploiement sur le Google Play Store